

Score-Based Generative Models

SMLD and DDPM

Score matching with Langevin dynamics → denoising diffusion probabilistic models

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{t, \mathbf{x}_0, \varepsilon} [\|\varepsilon - \varepsilon_{\theta}(\mathbf{x}_t, t)\|^2]$$

Sudarshan Babu

Slides created using Claude and Manus AI — each slide was prompted by the author.

Overview

1 Score Matching with Langevin Dynamics (SMLD)

Multi-scale scores, the noise-conditional score network, and annealed Langevin sampling.

2 Why Noise Helps

The manifold, scarcity, and mixing problems — and how perturbation fixes each.

3 The Limits of SMLD

Unstable training targets and slow, fragile MCMC sampling — the motivation for DDPM.

4 Denoising Diffusion (DDPM)

An explicit forward Markov chain, ϵ -prediction, and a learned Gaussian reverse step.

5 The Variational Objective

The ELBO, its KL decomposition, and the tractable reverse posterior.

Appendix — full reverse-process and ELBO derivations.

PART 01

Score Matching with Langevin Dynamics

Multi-scale scores · NCSN training · Annealed Langevin

Recap: What We Are Trying to Do

CENTRAL GOAL

Generate new samples by learning the gradient of the log data density (the **score**).

$$s_{\theta}(\mathbf{x}) \approx \nabla_{\mathbf{x}} \log p_{\text{data}}(\mathbf{x})$$

SAMPLING MECHANISM

Once learned, we generate samples via **Langevin dynamics**:

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \eta s_{\theta}(\mathbf{x}_k) + \sqrt{2\eta} \mathbf{z}_k$$

KEY INSIGHT

Score-based modeling replaces intractable density estimation with tractable gradient-field estimation.

Why Perturb the Data?

Before adding any noise, plain score matching breaks in three ways:

THE MANIFOLD PROBLEM

The score $\nabla \log p$ is undefined off the data manifold, so Langevin is ill-posed in the empty ambient space.

THE SCARCITY PROBLEM

Low-density regions contain no samples, so the network never learns a valid score there.

THE MIXING PROBLEM

Langevin gets trapped in a single mode and cannot move between well-separated modes.

The fix: perturb the data with Gaussian noise — at multiple scales — so the score is defined everywhere and sampling can mix.

We return to each problem in depth in Motivations 1–3.

The Noise Model: Perturbing the Data

1. THE PERTURBATION KERNEL

$$q_{\sigma}(\tilde{\mathbf{x}} \mid \mathbf{x}) = \mathcal{N}(\tilde{\mathbf{x}} \mid \mathbf{x}, \sigma^2 I)$$

Add isotropic Gaussian noise to clean data $\mathbf{x}$ to produce a noisy observation $\tilde{\mathbf{x}}$. The level σ controls the degree of corruption.

FORWARD NOISING PROCESS



Two images diffused to shared latents at $t \in \{250, 500, 750, 1000\}$.

2. THE NOISE SEQUENCE

$$\sigma_1 > \sigma_2 > \cdots > \sigma_L$$

A geometric sequence of noise levels defines a family of perturbed distributions $\{p_{\sigma}\}$.

Reading left-to-right is the forward process (increasing σ); right-to-left is the reverse (denoising). The score network must learn to navigate this entire spectrum.

Constructing Noisy Samples: The Sampling Recipe

$$\tilde{\mathbf{x}} = \mathbf{x} + \sigma \cdot \mathbf{z}, \quad \mathbf{z} \sim \mathcal{N}(0, I)$$

Equivalently:

$$\tilde{\mathbf{x}} \mid \mathbf{x} \sim \mathcal{N}(\mathbf{x}, \sigma^2 I)$$

NETWORK INPUT

$\tilde{\mathbf{x}}$ is the noisy sample fed to the score network $s\theta(\tilde{\mathbf{x}}, \sigma)$.

SCORE TARGET

The true score points back toward the clean data:

$$\nabla_{\tilde{\mathbf{x}}} \log p_{\sigma}(\tilde{\mathbf{x}} \mid \mathbf{x}) = -\frac{\tilde{\mathbf{x}} - \mathbf{x}}{\sigma^2}$$

MULTIPLE LEVELS

We repeat this for each σ in the schedule $\sigma_1 > \dots > \sigma_L$, creating a family of perturbed datasets.

Noising as Convolution: Fluffing the Manifold

$$p_{\sigma}(\tilde{\mathbf{x}}) = p_{\text{data}}(\mathbf{x}) * \mathcal{N}(0, \sigma^2 I) = \int p_{\text{data}}(\mathbf{x}) \mathcal{N}(\tilde{\mathbf{x}} \mid \mathbf{x}, \sigma^2 I) d\mathbf{x}$$

THE PROBLEM WITHOUT NOISE

Real data lies on a low-dimensional manifold $M \subset \mathbb{R}^D$. The density is concentrated on M and zero elsewhere, so $\nabla \log p_{\text{data}}$ is **undefined off the manifold** — Langevin dynamics is ill-posed.

THE FLUFFING EFFECT

Convolving with $\mathcal{N}(0, \sigma^2 I)$ fattens the manifold into a σ -thick tube. The perturbed density has **full support over $\mathbb{R}^D$** , so the score is well-defined everywhere — a valid target.

Larger $\sigma \rightarrow$ more fluffing, smoother density • Smaller $\sigma \rightarrow$ tighter tube, closer to the true manifold

The SMLD Approach

1. TRAINING (NCSN)

Learn a **Noise-Conditional Score Network** conditioned on the noise level σ :

$$s_{\theta}(\mathbf{x}, \sigma) \approx \nabla_{\mathbf{x}} \log p_{\sigma}(\mathbf{x})$$

For a geometric sequence of noise levels:

$$\sigma_1 > \sigma_2 > \dots > \sigma_L$$

2. SAMPLING (ANNEALED)

Run **Annealed Langevin Dynamics** sequentially.

Start at σ_1 (large noise) and anneal down to σ_L (small noise).

Each level initializes the next.

WHY?

This multi-scale design solves two critical failures of naive score matching.

The Unified Solution: Multi-Scale Scores

SINGLE SCALE (NAIVE)

$$\nabla_{\mathbf{x}} \log p(\mathbf{x})$$

Fails: undefined on the manifold, and poor mixing in low-density regions.

MULTI-SCALE (SMLD)

$$\nabla_{\mathbf{x}} \log p_{\sigma}(\mathbf{x}) \quad \text{for many } \sigma$$

Succeeds: large σ fills space for mixing; small σ recovers data.
Tractable conditional score:

$$-\frac{\tilde{\mathbf{x}} - \mathbf{x}}{\sigma^2}$$

$$\sigma_1 > \sigma_2 > \dots > \sigma_L$$

Global Exploration → Local Refinement (Smooth → Sharp)

Motivation 1: The Manifold Problem

THE ISSUE

Real-world data lies on a low-dimensional manifold embedded in high-dimensional space. The density drops to zero outside it, so $\nabla \log p(x)$ is undefined in the ambient space.

THE SMLD FIX

Perturb with noise: adding Gaussian noise “fluffs” the manifold. The perturbed $p\sigma(x)$ has full support everywhere, so the score becomes well-defined.

EMPIRICAL EVIDENCE

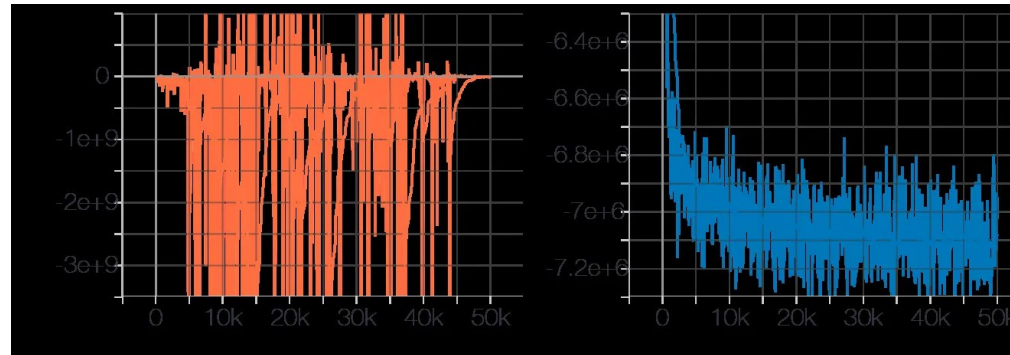


Fig 1: Without noise, training diverges (left); with noise, it stabilizes (right).

Motivation 2: The Scarcity Problem

LOW-DENSITY REGIONS

Score matching minimizes expected error over the data distribution. In regions where $p_{\text{data}}(x) \approx 0$ there are no samples, so the network **never learns the score in these empty spaces.**

THE SMLD FIX

Large noise levels (σ_1): a large σ convolves the data distribution, spreading probability mass into low-density regions. This provides a valid training signal everywhere, guiding the sampler toward the manifold.

Motivation 3: The Mixing Problem

ISOLATED MODES

Multimodal data has modes separated by vast low-density regions, so standard Langevin gets **trapped in a single mode** and can't mix between them.

WHY MIXING MATTERS

Without mixing, a run samples only one mode — so the modes appear in the **wrong proportions**, not the true distribution.

THE SMLD FIX

Annealing (large $\rightarrow$ small σ): large σ blurs the modes into one connected blob (easy to mix); anneal down to refine.

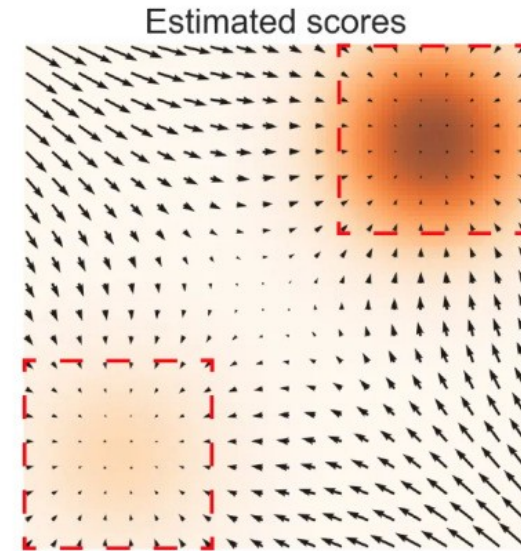


Fig 2: Estimated score field for a two-mode distribution.

Intuition: scores are accurate inside each mode but ≈ 0 in the gap — so Langevin can't cross between modes.

Training the Multi-Scale Score

SINGLE NOISE LEVEL — DENOISING SCORE MATCHING

$$\ell(\theta; \sigma) \triangleq \frac{1}{2} \mathbb{E}_{p_{\text{data}}} \mathbb{E}_{\tilde{\mathbf{x}}} \left[\left\| s_{\theta}(\tilde{\mathbf{x}}, \sigma) + \frac{\tilde{\mathbf{x}} - \mathbf{x}}{\sigma^2} \right\|^2 \right]$$

- Tractable: no adversarial training or sampling loop.
- Scalable: one U-Net learns all noise levels.
- Optimal: minimizer satisfies $s_{\theta^*} \approx \nabla \log p_{\sigma}$.

UNIFIED OBJECTIVE

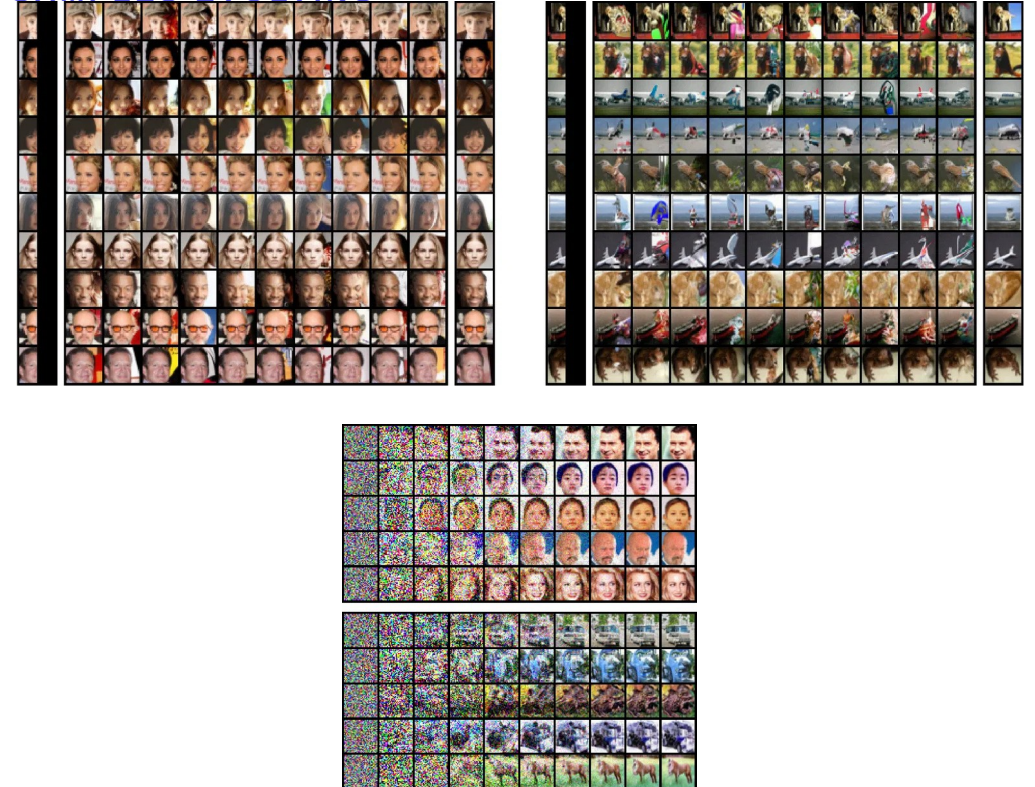
$$\mathcal{L}(\theta; \{\sigma_i\}_{i=1}^L) = \frac{1}{L} \sum_{i=1}^L \lambda(\sigma_i) \ell(\theta; \sigma_i)$$

Sampling with Annealed Langevin

ALGORITHM

```
Input: {sigma_i}, eps, T
1. init  x ~ N(0, I)
2. for i = 1 to L do
    a_i <- eps * sigma_i^2 / sigma_L^2  # anneal
    for t = 1 to T do
        z ~ N(0, I)
        x <- x + (a_i/2) s_th(x, sigma_i)
              + sqrt(a_i) z          # Langevin
    end for
3. end for
Return  x
```

SAMPLES EVOLVING



Figs 4–5: noise $\rightarrow$ structure as σ decreases.

Implementation Detail 1: Loss Weighting

THE PROBLEM

Empirically, the score norm scales inversely with noise:

$$\|s_{\theta}(\mathbf{x}, \sigma)\| \approx 1/\sigma$$

Unweighted loss:

$$\ell(\theta; \sigma) \propto 1/\sigma^2$$

Small- σ terms explode; gradients are dominated by the finest scale and coarse scales are ignored.

THE SOLUTION

Choose a weighting function:

$$\lambda(\sigma) = \sigma^2$$

Weighted loss:

$$\lambda(\sigma) \ell(\theta; \sigma) \propto \sigma^2 \cdot \frac{1}{\sigma^2} = 1$$

Balanced gradient contributions across all noise levels.

Implementation Detail 2: Step Size

$$\tilde{\mathbf{x}} \leftarrow \tilde{\mathbf{x}} + \underbrace{\frac{\alpha_i}{2} s_{\theta}(\tilde{\mathbf{x}}, \sigma_i)}_{\text{signal (drift)}} + \underbrace{\sqrt{\alpha_i} \mathbf{z}}_{\text{noise}}, \quad \alpha_i = \epsilon \sigma_i^2 / \sigma_L^2$$

SIGNAL MAGNITUDE

$$\text{Drift} \approx \frac{\alpha_i}{2} \|s_{\theta}\| \propto \frac{\alpha_i}{\sigma_i}$$

NOISE MAGNITUDE

$$\text{Noise} \approx \sqrt{\alpha_i}$$

SIGNAL-TO-NOISE

$$\frac{\sqrt{\alpha_i}}{\sigma_i} = \text{const}$$

Setting $\alpha_i \propto \sigma_i^2$ keeps the SNR constant — ensuring stable mixing dynamics across all noise levels.

Conceptual Flow & Results

THE LOGIC OF SMLD

1

The Goal

Generate samples via Langevin dynamics on learned scores.

2

The Method

Learn a family of scores (NCSN) and anneal noise levels.

3

The Motivation

Solves the manifold problem and the mixing problem.

4

The Outcome

High-fidelity samples competitive with GANs, no adversarial training.

CIFAR-10 UNCONDITIONAL RESULTS

Model	IS $\uparrow$	FID $\downarrow$
PixelCNN	4.60	65.93
WGAN-GP	7.86	36.4
SNGAN	8.22	21.7
ProgressiveGAN	8.80	—
NCSN (Ours)	8.87 ± 0.12	25.32

Table 1 — CIFAR-10 unconditional results

PART 02

Why SMLD Falls Short

Training instabilities · Fragile MCMC sampling · Motivation for DDPM

SMLD: What We Are Doing

TRAINING OBJECTIVE

$$s_{\theta}(\mathbf{x}, \sigma) \approx \nabla_{\mathbf{x}} \log p_{\sigma}(\mathbf{x})$$

with corruption

$$\mathbf{x} = \mathbf{x}_0 + \sigma \varepsilon$$

ANNEALED LANGEVIN SAMPLING

$$\mathbf{x} \leftarrow \mathbf{x} + \alpha s_{\theta}(\mathbf{x}, \sigma) + \sqrt{2\alpha} \mathbf{z}$$

It works in principle. But there are practical issues.

Training Dynamics Issue #1: Scale Mismatch

In SMLD, $x = x_0 + \sigma \epsilon$. As σ grows:

QUANTITY	SCALING WITH σ	IMPACT
Input Norm $\ x\ $	↑ Increases ($\approx \sigma$)	Network sees huge inputs
Target Norm $\ \nabla \log p\ $	↓ Decreases ($\approx 1/\sigma$)	Target signal vanishes
Gradient Magnitude	↓ Decreases	Loss dominated by small σ

THE RESULT — POOR CONDITIONING

Optimization is unbalanced. We are forced to use the heuristic weighting $\lambda(\sigma) = \sigma^2$ just to make gradients comparable.

Training Dynamics Issue #2: Activation Regime Drift

THE ROOT CAUSE

Input variance scales with σ^2 . The network sees inputs ranging from “clean” all the way to “massive noise.”

INTERNAL SHIFT

Activations operate at drastically different magnitudes. LayerNorm and residual scaling behave inconsistently across noise levels.

THE CONSEQUENCE

We are effectively training “many differently-scaled networks” with shared parameters — which complicates tuning and slows convergence.

Sampling Issue #1: MCMC Dependence

THE INNER LOOP

```
for i = 1 to L:  
  for t = 1 to T:      # many steps  
    z ~ N(0, I)  
    x = x + a*s(x) + sqrt(2a)*z  
  end  
end
```

WHY IT'S FRAGILE

- Requires many Langevin steps per level.
- Step size requires careful tuning.
- Approximate score $\Rightarrow$ biased drift.
- Sampling is expensive and fragile.

Hundreds or thousands of iterations are needed for convergence.

Sampling Issue #2: No Explicit Forward Markov Model

WHAT WE HAVE

$$p_{\sigma} = p_{\text{data}} * \mathcal{N}(0, \sigma^2 I)$$

SMLD defines only a set of marginal distributions — blurred versions of the data.

WHAT IS MISSING

- ✗ No explicit joint $q(x_{0:T})$.
- ✗ No closed-form reverse transition.

Reverse dynamics rely purely on MCMC correction (Langevin) — we lack a principled ancestral reverse model like $p(x_{t-1} | x_t)$.

Sampling Issue #3: Hyperparameter Sensitivity

The Noise Ladder

Geometric vs arithmetic schedule?

Step Sizes α

Per-level Langevin step size.

Langevin Steps / Level

How many T iterations each.

Loss Weights $\lambda(\sigma)$

Balancing across noise scales.

Small changes can destabilize training or sampling. Scaling to large, high-resolution models becomes extremely difficult.

Summary of SMLD Issues

TRAINING

- ✗ Input scale grows with σ .
- ✗ Target scale shrinks with σ .
- ✗ Gradients poorly conditioned.
- ✗ Heavy manual reweighting required.

SAMPLING

- ✗ MCMC inner loop (slow).
- ✗ Slow convergence.
- ✗ Hyperparameter sensitive.
- ✗ No explicit reverse Gaussian step.

What We Want Instead

- ✓ Stable input variance across noise levels
- ✓ Stable target magnitude
- ✓ Well-conditioned regression problem
- ✓ Explicit forward Markov chain
- ✓ Closed-form Gaussian reverse conditionals
- ✓ Fixed number of sampling steps

This motivates DDPM.

Reminder: What Was Broken in SMLD?

TRAINING

$$\mathbf{x} = \mathbf{x}_0 + \sigma \varepsilon$$

If σ is large, the input scale explodes; the target $s\theta \approx -\varepsilon/\sigma$ shrinks.

Net effect: poor conditioning, different activation regimes, and the manual weighting $\lambda(\sigma) = \sigma^2$.

SAMPLING

Annealed Langevin dynamics: many MCMC steps per σ , requiring step-size tuning and sensitive to drift errors.

Net effect: slow convergence, and no explicit reverse conditional to sample from.

PART 03

Denoising Diffusion (DDPM)

VP forward process · ϵ -prediction · Learned Gaussian reverse · The ELBO

DDPM Design Goal

Instead of “learn a score field and use Langevin to approximate reverse dynamics”...

Define an explicit forward Markov chain with simple Gaussian transitions, derive the exact reverse posterior inside that chain, then learn a reverse model that directly imitates those posteriors.

→ **This structural change removes inner-loop MCMC.**

Fix #1: Stabilize Variance — The VP Markov Forward Process

FORWARD STEP

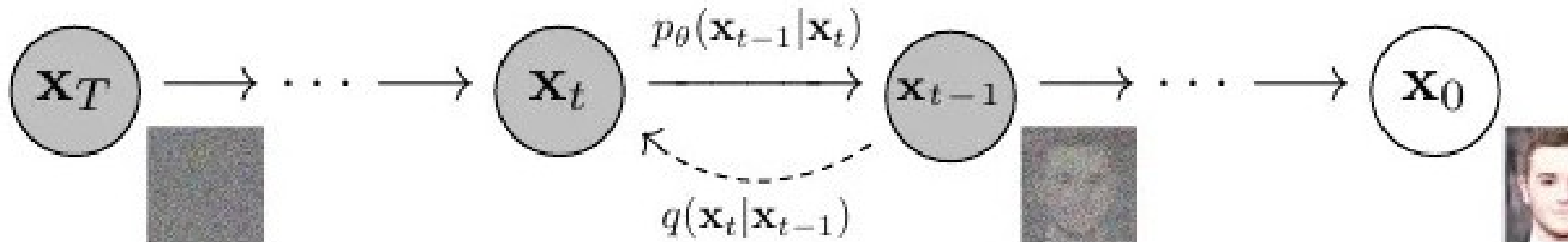
$$q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) = \mathcal{N}(\sqrt{\alpha_t} \mathbf{x}_{t-1}, \beta_t I)$$

SMLD: input norm grows as σ — activations drift, gradients shrink.

VARIANCE-PRESERVING CONSTRAINT

$$\alpha_t = 1 - \beta_t \Rightarrow \text{Var}[\mathbf{x}_t] \approx O(1)$$

DDPM: input norm stays $O(1)$ — consistent activations, stable gradients.



The forward process gradually adds noise; the reverse process learns to denoise.

Why Is the Forward Markov Process Essential?

1. TRACTABLE VARIATIONAL DISTRIBUTION

We can efficiently sample and evaluate the forward process because it factorizes:

$$q(\mathbf{x}_{1:T} \mid \mathbf{x}_0) = \prod_{t=1}^T q(\mathbf{x}_t \mid \mathbf{x}_{t-1})$$

2. CLOSED-FORM POSTERIOR

The reverse posterior $q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0)$ is Gaussian — it is the product of two Gaussians (the forward step and the marginal).

$$q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0)$$

Without the Markov + Gaussian structure, the KL decomposition isn't computable.

Closed-Form Marginal

Define the cumulative product $\bar{\alpha}_t = \prod \alpha_s$. The one-shot marginal is:

$$q(\mathbf{x}_t \mid \mathbf{x}_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0, (1 - \bar{\alpha}_t)I)$$

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I)$$

You can sample $\mathbf{x}_t$ for any t directly from $\mathbf{x}_0$ — no need to simulate intermediate steps. This enables efficient training.

Fix #2: Predict Noise Instead of Scaled Score

THE SMLD WAY

$$\text{Target} \approx -\varepsilon/\sigma$$

Target scale shrinks as σ grows. Gradients vanish at high noise unless manually reweighted.

THE DDPM WAY

$$\text{Predict } \varepsilon_{\theta}(\mathbf{x}_t, t) \approx \varepsilon$$

Target has constant scale (unit variance) for all t . **Gradients are uniform — far less reliance on weighting hacks.**

Fix #3: Replace MCMC with a Learned Gaussian Reverse Step

LEARNED REVERSE STEP

$$p_{\theta}(\mathbf{x}_{t-1} \mid \mathbf{x}_t) = \mathcal{N}(\mu_{\theta}(\mathbf{x}_t, t), \sigma_t^2 I)$$

Sampling update (one step):

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \varepsilon_{\theta}(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$$

REVERSE PROCESS



PROPERTY

SMLD

DDPM

Steps per level

Many (inner loop)

One

Reverse model

MCMC (biased)

Exact Gaussian

Hyperparameters

Step size, T, N

Schedule only

[Full reverse-step derivation ↗](#)

The DDPM Generative Model

PRIOR

$$p(\mathbf{x}_T) = \mathcal{N}(0, I)$$

REVERSE CHAIN

$$p_{\theta}(\mathbf{x}_{t-1} \mid \mathbf{x}_t) = \mathcal{N}(\mu_{\theta}(\mathbf{x}_t, t), \Sigma_{\theta}(\mathbf{x}_t, t))$$

Goal: make $p_{\theta}(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$ match the true reverse behavior induced by the forward chain.

VLB / ELBO: The Probabilistic Objective

We want to maximize the likelihood $\log p_\theta(\mathbf{x}_0)$, but marginalizing all latents is intractable:

$$\log p_\theta(\mathbf{x}_0) = \log \int p_\theta(\mathbf{x}_{0:T}) d\mathbf{x}_{1:T}$$

Introduce the forward chain $q(\mathbf{x}_{1:T}|\mathbf{x}_0)$ as a variational distribution to derive the ELBO:

$$\log p_\theta(\mathbf{x}_0) \geq \mathbb{E}_q \left[\log \frac{p_\theta(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T} | \mathbf{x}_0)} \right]$$

Expand the ELBO

$$p_{\theta}(\mathbf{x}_{0:T}) = p(\mathbf{x}_T) \prod_{t=1}^T p_{\theta}(\mathbf{x}_{t-1} \mid \mathbf{x}_t) \quad q(\mathbf{x}_{1:T} \mid \mathbf{x}_0) = \prod_{t=1}^T q(\mathbf{x}_t \mid \mathbf{x}_{t-1})$$

Substituting these into the ELBO yields:

$$\mathbb{E}_q \left[\log p(\mathbf{x}_T) + \sum_{t=1}^T \log p_{\theta}(\mathbf{x}_{t-1} \mid \mathbf{x}_t) - \sum_{t=1}^T \log q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) \right]$$

Rearranging gives a sum of KL-divergence terms.

KL Decomposition

$$\mathbb{E}_q \left[D_{\text{KL}}(q(\mathbf{x}_T \mid \mathbf{x}_0) \parallel p(\mathbf{x}_T)) + \sum_{t=2}^T D_{\text{KL}}(q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) \parallel p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t)) - \log p_\theta(\mathbf{x}_0 \mid \mathbf{x}_1) \right]$$

So the main learning problem becomes: **match** $p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$ to the true posterior $q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0)$.

Because everything is Gaussian, these KL divergences are tractable.

Why We Can't Use $q(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$ Directly

We'd like $q(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$, but:

$$q(\mathbf{x}_{t-1} \mid \mathbf{x}_t) = \int q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) q(\mathbf{x}_0 \mid \mathbf{x}_t) d\mathbf{x}_0$$

That's a mixture of Gaussians over $\mathbf{x}_0$, and mixtures are not Gaussian in general $\Rightarrow$ intractable.

This is the key reason DDPM introduces $\mathbf{x}_0$ in the derivation.

Key Trick: Condition on $\mathbf{x}_0$

Instead, compute the posterior inside the known forward model:

$$q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0)$$

BECAUSE THE FORWARD PROCESS IS LINEAR-GAUSSIAN MARKOV

- ✓ This posterior will be Gaussian.
- ✓ It can be derived exactly by multiplying Gaussians.

The Practical Reverse Step Mean

After substituting and simplifying, the reverse mean is:

$$\mu_{\theta}(\mathbf{x}_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \varepsilon_{\theta}(\mathbf{x}_t, t) \right)$$

This formula tells us exactly how to denoise $\mathbf{x}_t$.

Notice the similarity to Langevin dynamics?

The Final Sampling Step

To sample $\mathbf{x}_{t-1}$ from $\mathbf{x}_t$:

$$\mathbf{x}_{t-1} = \mu_{\theta}(\mathbf{x}_t, t) + \sigma_t \mathbf{z}, \quad \mathbf{z} \sim \mathcal{N}(0, I)$$

This is a single Gaussian step. Iterate from $T \rightarrow 0$. No MCMC. No inner loops.

DDPM Algorithm Recap

TRAINING

1. Sample $x_0 \sim q(x_0)$, $\varepsilon \sim N(0, I)$, $t \sim \text{Uniform}(1, T)$.
2. Take a gradient step on:

$$\left\| \varepsilon - \varepsilon_{\theta}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon, t) \right\|^2$$

SAMPLING

1. Sample $x_T \sim N(0, I)$.
2. For $t = T, \dots, 1$:

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \varepsilon_{\theta}(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$$

3. Return x_0 .

Loss Function Derivation (Simplified)

The KL term becomes a weighted MSE between the true noise and the predicted noise:

$$\mathbb{E} \left[\frac{\beta_t^2}{2\sigma_t^2 \alpha_t (1 - \bar{\alpha}_t)} \|\varepsilon - \varepsilon_\theta\|^2 \right]$$

DDPM ignores the weighting term for better sample quality:

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{t, \mathbf{x}_0, \varepsilon} [\|\varepsilon - \varepsilon_\theta(\mathbf{x}_t, t)\|^2]$$

DDPM vs. SMLD: The Final Verdict

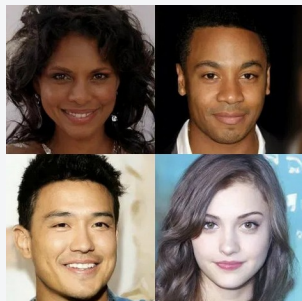
SMLD

- ✗ Implicit reverse process (Langevin approximation)
- ✗ Unstable training targets — scale varies with σ
- ✗ Slow, fragile MCMC sampling with inner loops

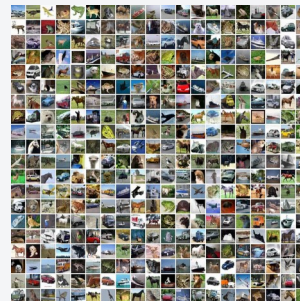
DDPM

- ✓ Explicit reverse process (closed-form Gaussian posterior)
- ✓ Stable training targets — unit-variance noise prediction
- ✓ Fixed-step ancestral sampling — no inner loops

DDPM GENERATION QUALITY



CelebA-HQ 256×256



CIFAR-10 — FID 3.17 / IS 9.46



Figure 3: LSUN Church samples. FID=7.89

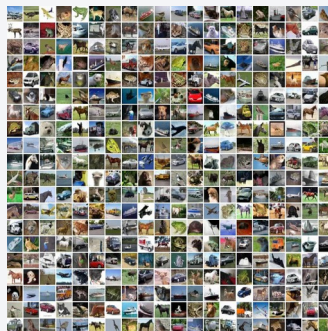
Figure 4: LSUN Bedroom samples. FID=4.90

LSUN Church 7.89 / Bedroom 4.90

DDPM Generation Quality



CelebA-HQ 256×256



CIFAR-10 — FID 3.17

QUANTITATIVE RESULTS (CIFAR-10 UNCONDITIONAL)

Model	IS $\uparrow$	FID $\downarrow$	NLL (bits/dim) $\downarrow$
PixelCNN	4.60	65.93	3.03
NCSN (Song & Ermon '19)	8.87 ± 0.12	25.32	—
SNGAN	8.22 ± 0.05	21.7	—
StyleGAN2 + ADA	9.74 ± 0.05	3.26	—
DDPM (L_{simple}) [Ours]	9.46 ± 0.11	3.17	≤ 3.75

A P P E N D I X

Appendix

The DDPM reverse process, step by step

Bayes & the posterior · Completing the square · Mean & variance · ELBO derivation

Reverse Step — What We Want, and Why It's Hard

Ideally we'd sample with the true reverse conditional $q(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$. But marginally it is a mixture over the unknown $\mathbf{x}_0$:

$$q(\mathbf{x}_{t-1} \mid \mathbf{x}_t) = \int q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) q(\mathbf{x}_0 \mid \mathbf{x}_t) d\mathbf{x}_0$$

A mixture of Gaussians is not Gaussian $\Rightarrow$ intractable. Fix: condition on $\mathbf{x}_0$ for a closed-form posterior, then replace $\mathbf{x}_0$ with a model estimate.

Bayes' Rule, Exactly

The forward Markov property gives $q(\mathbf{x}_t \mid \mathbf{x}_{t-1}, \mathbf{x}_0) = q(\mathbf{x}_t \mid \mathbf{x}_{t-1})$, so:

$$\begin{aligned} q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) &= \frac{q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) q(\mathbf{x}_{t-1} \mid \mathbf{x}_0)}{q(\mathbf{x}_t \mid \mathbf{x}_0)} \\ &\propto q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) q(\mathbf{x}_{t-1} \mid \mathbf{x}_0) \end{aligned}$$

Still Gaussian: the denominator $q(\mathbf{x}_t \mid \mathbf{x}_0)$ doesn't depend on $\mathbf{x}_{t-1}$, so it is only a normalizing constant.

The Two Gaussian Factors

Both factors are Gaussian in the same variable $\mathbf{x}_{t-1}$:

$$\text{Likelihood: } q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) = \mathcal{N}(\sqrt{\alpha_t} \mathbf{x}_{t-1}, \beta_t I)$$

$$\text{Prior: } q(\mathbf{x}_{t-1} \mid \mathbf{x}_0) = \mathcal{N}(\sqrt{\bar{\alpha}_{t-1}} \mathbf{x}_0, (1 - \bar{\alpha}_{t-1})I)$$

log-Gaussian is quadratic; the sum of quadratics is quadratic; exp of a quadratic is Gaussian.

Collect into a Quadratic Form

Adding the exponents and keeping the terms in $\mathbf{x}_{t-1}$ gives $-\frac{1}{2}(\mathbf{x}^T \mathbf{A} \mathbf{x} - 2\mathbf{b}^T \mathbf{x} + \text{const})$, with:

$$A = \frac{\alpha_t}{\beta_t} + \frac{1}{1 - \bar{\alpha}_{t-1}}$$
$$b = \frac{\sqrt{\alpha_t}}{\beta_t} \mathbf{x}_t + \frac{\sqrt{\bar{\alpha}_{t-1}}}{1 - \bar{\alpha}_{t-1}} \mathbf{x}_0$$

The square roots matter: the forward mean is $\sqrt{\alpha_t} \mathbf{x}_{t-1}$ and the marginal mean is $\sqrt{\bar{\alpha}_{t-1}} \mathbf{x}_0$.

Complete the Square → Gaussian Posterior

A quadratic $-\frac{1}{2}(\mathbf{x}^T \mathbf{A} \mathbf{x} - 2\mathbf{b}^T \mathbf{x})$ is a Gaussian with precision $\mathbf{A}$, covariance $\mathbf{A}^{-1}$, mean $\mathbf{A}^{-1}\mathbf{b}$:

$$q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}(\tilde{\mu}_t, \tilde{\beta}_t I), \quad \tilde{\mu}_t = \mathbf{A}^{-1} \mathbf{b}, \quad \tilde{\beta}_t = \mathbf{A}^{-1}$$

$\mathbf{A}$ = precision (inverse variance); $\tilde{\beta}_t = \mathbf{A}^{-1}$ = variance; $\sqrt{\tilde{\beta}_t}$ = the std used for the sampling noise.

Posterior Variance β_t

Invert the precision and use $\beta_t = 1 - \alpha_t$, $\bar{\alpha}_t = \alpha_t \bar{\alpha}_{t-1}$ (so the denominator collapses to $1 - \bar{\alpha}_t$):

$$A^{-1} = \frac{\beta_t(1 - \bar{\alpha}_{t-1})}{\alpha_t(1 - \bar{\alpha}_{t-1}) + \beta_t}$$
$$\tilde{\beta}_t = A^{-1} = \beta_t \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t}$$

A fixed constant from the noise schedule — no learnable parameters.

Posterior Mean (given $\mathbf{x}_0$)

Carrying out $A^{-1}b$ and simplifying gives the exact mean inside the forward process:

$$\tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0) = \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} \mathbf{x}_t + \frac{\beta_t \sqrt{\bar{\alpha}_{t-1}}}{1 - \bar{\alpha}_t} \mathbf{x}_0$$

Exact identity — but it needs $\mathbf{x}_0$, which we don't have at sampling time.

Estimate $\mathbf{x}_0$ via Predicted Noise

Invert the forward marginal $\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon$ and use the network's noise prediction:

$$\hat{\mathbf{x}}_0(\mathbf{x}_t, t) = \frac{1}{\sqrt{\bar{\alpha}_t}} \left(\mathbf{x}_t - \sqrt{1 - \bar{\alpha}_t} \varepsilon_{\theta}(\mathbf{x}_t, t) \right)$$

Why not just re-noise $\hat{\mathbf{x}}_0$? that samples the prior $q(\mathbf{x}_{t-1}|\mathbf{x}_0)$ and ignores the realized $\mathbf{x}_t$ — too much variance. The posterior conditions on $\mathbf{x}_t$ (variance β_t).

Practical Mean & Ancestral Sampling

Substitute $\mathbf{x}_0 \approx \hat{\mathbf{x}}_0$ into $\mu_{\tilde{\epsilon}}$ to get the noise-prediction mean, then take one Gaussian step:

$$\mu_{\theta}(\mathbf{x}_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \varepsilon_{\theta}(\mathbf{x}_t, t) \right)$$

$$\mathbf{x}_{t-1} = \mu_{\theta}(\mathbf{x}_t, t) + \sqrt{\tilde{\beta}_t} \mathbf{z}, \quad \mathbf{z} \sim \mathcal{N}(0, I)$$

Set $\mathbf{z} = 0$ for the final step ($t = 1 \rightarrow 0$). No inner-loop MCMC.

ELBO Derivation: Multiply / Divide Trick

Start with the log-likelihood of the data $\mathbf{x}_0$, then multiply and divide by $q(\mathbf{x}_{1:T}|\mathbf{x}_0)$:

$$\begin{aligned}\log p_{\theta}(\mathbf{x}_0) &= \log \int p_{\theta}(\mathbf{x}_{0:T}) d\mathbf{x}_{1:T} \\ &= \log \int q(\mathbf{x}_{1:T} \mid \mathbf{x}_0) \frac{p_{\theta}(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T} \mid \mathbf{x}_0)} d\mathbf{x}_{1:T}\end{aligned}$$

ELBO Derivation: Jensen's Inequality

Applying Jensen's inequality ($\log \mathbb{E} \geq \mathbb{E} \log$):

$$\log p_{\theta}(\mathbf{x}_0) \geq \mathbb{E}_{q(\mathbf{x}_{1:T}|\mathbf{x}_0)} \left[\log \frac{p_{\theta}(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T} | \mathbf{x}_0)} \right]$$

$$L_{\text{ELBO}} = \mathbb{E}_q \left[\log p_{\theta}(\mathbf{x}_{0:T}) - \log q(\mathbf{x}_{1:T} | \mathbf{x}_0) \right]$$

ELBO Derivation: Expanding Terms

Use the Markov property to expand the log-joint and the log-variational posterior:

$$\log p_{\theta}(\mathbf{x}_{0:T}) = \log p(\mathbf{x}_T) + \sum_{t=1}^T \log p_{\theta}(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$$
$$\log q(\mathbf{x}_{1:T} \mid \mathbf{x}_0) = \sum_{t=1}^T \log q(\mathbf{x}_t \mid \mathbf{x}_{t-1})$$

ELBO Derivation: Final Form

Combining the terms, the ELBO becomes:

$$L_{\text{ELBO}} = \mathbb{E}_q \left[\log p(\mathbf{x}_T) + \sum_{t=1}^T \log p_{\theta}(\mathbf{x}_{t-1} \mid \mathbf{x}_t) - \sum_{t=1}^T \log q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) \right]$$

We then rearrange this into the sum of KL divergences.

References

- [1] **Song & Ermon (2019).** *Generative Modeling by Estimating Gradients of the Data Distribution.* NeurIPS.
- [2] **Ho, Jain & Abbeel (2020).** *Denoising Diffusion Probabilistic Models.* NeurIPS.
- [3] **Sohl-Dickstein, Weiss, Maheswaranathan & Ganguli (2015).** *Deep Unsupervised Learning using Nonequilibrium Thermodynamics.* ICML.
- [4] **Welling & Teh (2011).** *Bayesian Learning via Stochastic Gradient Langevin Dynamics.* ICML.
- [5] **Vincent (2011).** *A Connection Between Score Matching and Denoising Autoencoders.* Neural Computation.
- [6] **Hyvärinen (2005).** *Estimation of Non-Normalized Statistical Models by Score Matching.* JMLR.
- [7] **Song, Sohl-Dickstein, Kingma, Kumar, Ermon & Poole (2021).** *Score-Based Generative Modeling through Stochastic Differential Equations.* ICLR.